
Software Requirements Specification

for

Gordon Ramsay Restaurant Reservation

Version 1.4

Prepared by

	Group Name:	Qdreon Lab Team 1
Arcilla, Reuel Matthew	24101057	24101057@usc.edu.ph
Belocura, Kisha Faye	24100894	24100894@usc.edu.ph
Cantago, Dale Ryoko Jose	22600110	22600110@usc.edu.ph
Castroverde, Mickel Angelo	24102411	24102411@usc.edu.ph
Durango, Cyrus Alain	24104754	24104754@usc.edu.ph

Instructor:	Godwin S. Monserate, PhD CCNA/CCAI, MTCNA, MTCRE, MTCUME Engr. Thairell C. Imperial, MCPE
Course:	CPE 2201 - Software Design
Lab Section:	LBCEA2TC TC
Date:	April 30, 2026

1 Introduction	1
1.1 Document Purpose	1
1.2 Product Scope	1
1.3 Intended Audience and Document Overview	1
1.4 Definitions, Acronyms, and Abbreviations	2
1.5 Document Conventions	3
1.6 References and Acknowledgments	3
2 Overall Description	4
2.1 Product Overview	4
2.2 Product Functionality	5
2.3 Design and Implementation Constraints	6
2.4 Assumptions and Dependencies	7
3 Specific Requirements	8
3.1 External Interface Requirements	8
3.1.1 User Interfaces	8
3.1.2 Hardware Interfaces	8
3.1.3 Software Interfaces	8
3.2 Functional Requirements	9
3.2.1 Customer Portal Functions	9
3.2.2 Restaurant Management System (Admin)	9
3.3 Use Case Model	10
3.3.1 Use Case #1 (U1): Book a Table & Pay Deposit	11
3.3.2 Use Case #2 (U2): Update Table Status via Static Grid	12
3.3.3 Use Case #3 (U3): Join Virtual Waitlist	13
3.3.4 Use Case #4 (U4): Manage Account & Login	14
3.3.5 Use Case #5 (U5): Manage Reservations & Shifts	15
3.3.6 Use Case #6 (U6): View Guest CRM Data	16
4 Other Non-functional Requirements	17
4.1 Performance Requirements	17
4.2 Safety and Security Requirements	17
4.3 Software Quality Attributes	18
5 Other Requirements	19

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Arcilla, Reuel Matthew Belocura, Kisha Faye Cantago, Dale Ryoko Jose	Initial draft including the raw, un-negotiated requirements from the stakeholders (LT2)	02/13/26
1.1	Castroverde, Mickel Angelo Arcilla, Reuel Matthew Belocura, Kisha Faye Durango, Cyrus Alain	Revised scope to reflect the negotiated MVP, including Single-Tenant Management, conversion of SMS to Email notifications, and replacement of the drag-and-drop editor with a static, interactive floor plan grid, per stakeholder agreement.	02/15/26
1.2	Castroverde, Mickel Angelo Durango, Cyrus Alain	Updated the Table of Contents. Resolved waitlist automation logic by adding Booking Cancellation functional requirements (FR-10) and a dedicated Use Case (U7). Added administrative Menu Management capabilities (FR-11). Updated the Data Dictionary to include 'Cancelled' reservation states, updated document versioning, and date. Added Actor Payment Gateway for Use Case (U1), and have it include (U7). Use Case Diagram is updated with these Connections and the updated document version and date.	04/16/26
1.3	Castroverde, Mickel Angelo	Updated functional requirements to include a 5-minute concurrency lock timeout with explicit failed-lock error handling (FR-3), a 12-pax maximum limit on table combinations (FR-4), and a 10-minute waitlist acceptance window (FR-5). Enhanced administrative and UI requirements by formally defining floor	04/21/26

Version	Primary Author(s)	Description of Version	Date Completed
		plan color-coding and automatic state synchronization (FR-7), input validation for operating hours (FR-8), a 15-minute automated no-show trigger (FR-9), manual waitlist overrides (FR-12), and system health monitoring (FR-13). Finally, updated system constraints to mandate UTC zone standardization (DB-3) and an automated 'Delete Account' function to ensure full compliance with the Right to Erasure (LEG-1).	
1.4	Castroverde, Mickel Angelo Durango, Cyrus Alain	Corrected UML diagram logic by converting the 'Cancel Reservation' relationship from <<includes>> to <<extends>> (Use Case #1). Added a 60-minute pre-closing cutoff constraint to the automated waitlist protocol (FR-5). Streamlined the Product Overview (Section 2.1) to remove redundant architectural descriptions.	04/30/26

1 Introduction

This section provides an overview of the Software Requirements Specification (SRS) for the Gordon Ramsay Restaurant Reservation System. It outlines the document's primary purpose, the complete scope of the proposed product, the intended audience, and the fundamental definitions and acronyms used throughout the specification.

1.1 Document Purpose

The purpose of this document is to formally specify the software requirements for the Restaurant Reservation System. It comprehensively defines the Minimum Viable Product (MVP) functional and non-functional requirements based on the negotiated agreements between the developer team and stakeholder team.

This SRS covers a single-tenant platform, detailing the necessary behaviors and constraints for two primary subsystems: the Customer Portal (for diners) and the Restaurant Management System (for restaurant operations).

1.2 Product Scope

The Gordon Ramsay Restaurant Reservation System is a web-based application designed to seamlessly connect diners with restaurant management for a single, dedicated restaurant location. The primary goal and objective of this software is to replace manual reservation logbooks with a scalable digital environment where the restaurant can optimize its daily operations and offer its customers a frictionless booking experience.

The scope of the product encompasses two main modules: a Customer Portal and a Restaurant Management System. For customers, the system delivers the benefit of an online discovery engine featuring integrated "view-only" menus, virtual waitlists, and automated Email notifications to eliminate double bookings. For restaurant administrators, the system provides robust operations control through a static but real-time interactive floor plan grid, Guest CRM tracking, and deposit handling capabilities.

1.3 Intended Audience and Document Overview

This document is intended for three primary audiences: the **client/stakeholders** (Group LT3), the professors and instructors evaluating the system design, and the **developer team** (Group LT1) responsible for analyzing the platform's technical feasibility.

The remainder of this SRS is organized into the following sections:

- **Section 2: Overall Description** provides a high-level overview of the product, including its general functionality, user characteristics, and business constraints, without delving into technical programming specifics.
- **Section 3: Specific Requirements** contains the exhaustive, technical list of all functional and non-functional requirements (such as the static floor plan grid, deposit handling, email notifications, and performance metrics) needed to build the Minimum Viable Product (MVP).

Suggested Reading Sequence:

- **For Client and Professors:** It is recommended to begin with the overview in **Section 1**, proceed through **Section 2** to understand the general business scope and constraints, and finally review **Section 3** to verify that specific feature requests and negotiated compromises have been accurately documented.
- **For Developers and Testers:** After grasping the overall project scope in Sections 1 and 2, technical staff should proceed to and focus heavily on **Section 3**, as it serves as the definitive blueprint for system architecture and feature implementation.

1.4 Definitions, Acronyms, and Abbreviations

ACID: Atomicity, Consistency, Isolation, Durability (Database transaction properties)

API: Application Programming Interface

BaaS: Backend-as-a-Service

COMET: Concurrent Object Modelling and Architectural Design Method

CRM: Customer Relationship Management

HTTPS/TLS: Hypertext Transfer Protocol Secure / Transport Layer Security

IEEE: Institute of Electrical and Electronics Engineers

MVP: Minimum Viable Product

PCI-DSS: Payment Card Industry Data Security Standard

PII: Personally Identifiable Information

RBAC: Role-Based Access Control

SMTP: Simple Mail Transfer Protocol

SRS: Software Requirements Specification

UML: Unified Modeling Language

1.5 Document Conventions

This document follows the IEEE formatting requirements for Software Requirements Specification. To ensure consistency and readability across the document, the following conventions have been established:

Formatting Conventions

- **Page Layout:** The document maintains standard 1-inch margins on all sides.
- **Typography:** The standard text is written in Arial font, size 11, and is single-spaced. Section and subsection titles follow the predefined hierarchical styling of the IEEE template.

Typographical & Linguistic Conventions

- **Italics:** Italicized text is strictly used for comments, explanatory notes, or to introduce hypothetical examples.
- **Bold Text:** Bold text is utilized to emphasize critical system constraints, high-priority MVP features, or to highlight primary actors within the use case descriptions.
- **Requirement Statements:** This document distinguishes between mandatory and optional features using specific terminology. The word “shall” dictates a mandatory, verifiable system requirement that must be implemented in the final product. The word “should” denotes a desirable feature that is not strictly mandatory for the Minimum Viable Product (MVP).

1.6 References and Acknowledgments

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998.

Gomaa, H. (2011). Software Modeling and Design: UML, Use Cases, Patterns, and Software Architectures (COMET Method). Cambridge University Press.

Group LT2 Stakeholder Requirements Overview (Initial vision and scope text document provided by RJ, Felix, Jharyd, Renz, and Sim), February 3, 2026.

Group LT1 & LT2 Initial Requirements Elicitation Meeting [Video Recording]. YouTube. Available at: <https://youtu.be/Sr2yzlQ4gug>

Group LT1 & LT2 Scope Negotiation and MVP Finalization Meeting [Video Recording]. YouTube. Available at: <https://youtu.be/nHiaiTB2a7w>

Object Management Group (OMG). (2017). Unified Modeling Language (UML) Specification Version 2.5.1. Available at: <https://www.omg.org/spec/UML/2.5.1/PDF>

2 Overall Description

2.1 Product Overview

The Gordon Ramsay Restaurant Reservation System is a web-based, single-tenant platform designed to modernize table booking and streamline operations for a specific, dedicated restaurant location. It is a standalone software solution that replaces manual reservation logbooks, phone-based scheduling, and disconnected spreadsheet systems with a centralized, real-time digital environment.

The system serves primary users: Customers (diners) and Restaurant Administrators (staff/managers). Customers use the web interface to search for the restaurant, check real-time availability, book tables, view the integrated menu, join virtual waitlists, and manage their profiles. To ensure reliable and cost-effective communication, all automated booking confirmations and reminders are handled exclusively through email notifications triggered by the backend server.

Restaurant Administrators access a dedicated operational dashboard to manage reservations, adjust operating hours, and monitor live table availability using a static, interactive floor plan grid. From this interface, staff can manually update booking statuses (such as seated, completed, no-show, or late) and log walk-ins or phone reservations. The system also features a built-in guest CRM to track past visits, flag VIPs, and log specific allergy notes for a more personalized dining experience. Lastly, it integrates a payment gateway to process upfront customer deposits, helping the restaurant cut down on financial losses from no-shows.

The application operates on a centralized architecture utilizing Supabase as a Backend-as-a-Service (BaaS). This infrastructure seamlessly integrates booking logic, availability management, secure user authentication, and real-time database synchronization to ensure accurate table scheduling and prevent double-booking. By consolidating reservation management, guest tracking, and operational oversight into one integrated system, the platform enhances the customer experience and improves restaurant efficiency.

2.2 Product Functionality

The Gordon Ramsay Restaurant is divided into two primary functional modules for the Minimum Viable Product (MVP):

Customer Portal Functions

- Customer Account Management
 - Allow users to create and manage personal accounts.
 - Store contact information and dietary preferences.
 - Maintain reservation history.
- Restaurant Search & Availability Viewing
 - Allow customers to view available reservation time slots in real time.
- Real-Time Booking
 - Used to facilitate instant table reservations, and may include automated conflict resolution to prevent double-booking
- Waitlist Management
 - Provide a digital queue for when tables or venues are booked, notifying of availability in case of cancellation.
 - Allow customers to join a digital waitlist when fully booked
 - Automatically manage and update waitlist status
- Automated Notifications:
 - Enable sending booking confirmations via email and push notifications.
 - Send reminder notifications before the reservation time.

Restaurant Management Functions

- Visual Table Management
 - Offers a static, interactive floor plan grid interface for staff to manage floor plans and real-time table statuses.
 - Update reservation status (e.g., Seated, Completed, No-Show, Late)
- Guest CRM
 - To track the customers' preferences, history, health-related notes, and personalized experiences
- Reservation Management (Admin Side)
 - View bookings in list and calendar formats.
 - Manually enter phone reservations and walk-ins.

2.3 Design and Implementation Constraints

The following constraints define technical, operational, and methodological boundaries that must be observed during the design and development.

1. Development Methodology Constraint

The system must follow the COMET (Concurrent Object Modelling and Architectural Design Method) method for software design. COMET will guide the identification of objects, subsystems, architectural layers, and communication mechanisms. This ensures a structured, object-oriented, and concurrent-aware system architecture.

Additionally, the system must use UML (Unified Modeling Language) for modeling and documentation. UML artifacts such as

- Use Case Diagrams
- Class Diagrams
- Sequence Diagrams

2. Technical and Performance Constraints

- a. **Latency:** To accommodate the “Real-Time Availability” feature, UI load time is 3 seconds, Concurrency lock resolution is 1 second, and Availability query response is 3 seconds max.
- b. **Resource:** Accommodate the mobile web component, making sure the website is usable across devices, even with limited memory, ensuring wide accessibility for a wide range of users.

3. Security Constraints

- a. **Data Protection:** Sensitive user data (password, contact details) must be securely stored using hashing and encryption mechanisms.
- b. **System Integrity:** The system must ensure data integrity and operational reliability by preventing double-booking, performing regular backups, and logging critical administrative actions for audit purposes.

4. Maintenance and Coding Standards

- a. **Language Requirements:** For long-term maintainability for the client, the codebase is written in JavaScript for the frontend and backend.
- b. **Modularity:** Developers must follow the SOLID principles of object-oriented design. Each module shall adhere to single-responsibility principles to improve maintainability, debugging, and future extensibility.
- c. **In-line Documentation:** All complex logic is to be commented on for the understanding of fellow developers.

2.4 Assumptions and Dependencies

- **Assumptions**

- **Connectivity**—The restaurant is assumed to have a high-speed internet connection to maintain the real-time features for the restaurant, specifically, the double booking prevention of the system.
- **Hardware**—Used by the restaurant management and provided by them to use the Visual Table Management system effectively as a host stand.
Touch Screen Terminal: iPad, Android Tablets, GSAN GS-T3/A5
- **Technical Literacy**—Staff and Management to possess basic digital literacy skills to navigate the management system and the dashboard and perform entries for walk-in customers.
- **Legal Compliance**—Assume the storage of customer data will be compliant with the Data Privacy Act of 2012 in the Philippines, and consent to those terms upon registration.
- **Single Source of Truth**—The restaurant must use this system as its exclusive reservation tool to ensure system effectiveness.

- **Dependencies**

- **Third-Party APIs Payment Gateway**: PayMongo, DragonPay, Maya Business, and HitPay
- **Communication Services**: Twilio, SendGrid, and Google Firebase Cloud Messaging.
- **Cross-Browser Compatibility**—makes sure specifically that the Customer Portal will support the web standards of browsers and is also usable on mobile operating systems such as IOS and Android
- **Identity Providers**—usage of “Login with Google,” which means the system is dependent on the uptime and API stability of those third-party authentication services.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

There are two main users who will interact with the Gordon Ramsay Restaurant Reservation System: the Customer (*diner*) and the Restaurant Administrator (*staff/owner*). The system shall provide a responsive, web-based interface that automatically adapts its layout for optimal viewing on desktop monitors, tablets, and mobile smartphone screens.

- Customer Interface: The customer portal shall present a clean, intuitive layout allowing users to search for table availability, view the integrated menu, join the waitlist, complete the simulated deposit checkout, and manage their personal profiles.
- Administrator Interface: The admin dashboard shall provide a secure, behind-the-scenes view featuring a calendar/list reservation manager, a Guest CRM table, and a static, interactive floor plan grid that uses color-coding to display real-time table statuses (e.g., *Green for Available, Red for Occupied*).

3.1.2 Hardware Interfaces

Since the Gordon Ramsay Restaurant Reservation System is a web-based application, it does not require any custom or proprietary hardware.

- The system shall operate efficiently on standard, internet-connected consumer devices, including Windows PCs, Apple Macs, tablets (e.g., iPads used by restaurant front-desk hosts), and smartphones.
- No direct integration with physical restaurant hardware (such as POS receipt printers or physical table sensors) is included in their MVP scope.

3.1.3 Software Interfaces

The system's architecture relies heavily on external software integrations to fulfill its core functionality for the MVP (*at the moment*):

- **Supabase (Backend-as-a-Service):** The application shall interface securely with Supabase via API to handle all PostgreSQL database queries (*storing reservations, floor plans, and CRM data*) and to manage user authentication (*secure login/registration*).
- **SMTP Email Provider:** The system shall interface with a standard SMTP Email API (triggered through the Supabase backend) to dispatch automated booking confirmations, calendar invites, and waitlist alerts to customers.

3.2 Functional Requirements

Functional requirements capture the intended behavior of the system, expressed as services, tasks, or functions the system is required to perform.

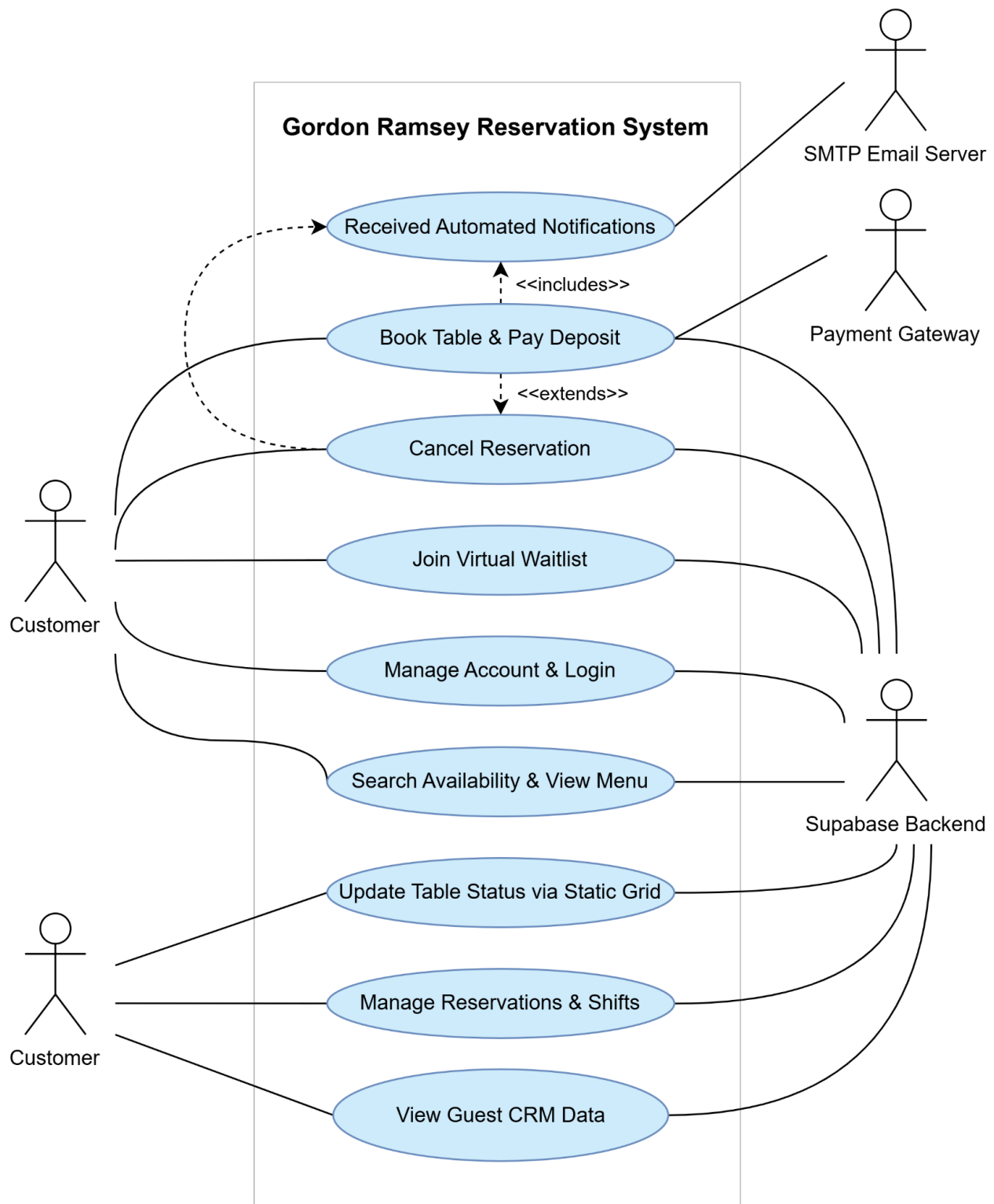
3.2.1 Customer Portal Functions

- **FR-1 (Account Management):** The system shall allow users to register, securely log in, and manage their profile details (including contact information and dietary restrictions) via the Supabase authentication backend. All unique identifiers (e.g., *Customer_ID*) shall be stored as UUIDs to ensure secure relational integrity
- **FR-2 (Search & Discovery):** The system shall allow customers to search for real-time table availability by specifying a Date, Time, and Party Size (Pax), and provide access to a view-only digital menu before booking.
- **FR-3 (Booking Engine):** The system shall securely lock an available table upon user selection and utilize a simulated checkout process to handle required reservation deposits. The system shall automatically release the database row-lock and revert the table status to 'Available' if the checkout is not completed within a 5-minute timeout window. If the 1-second concurrency lock fails due to a conflict, the system shall immediately abort the transaction and display a 'Table already reserved' error message to the customer, prompting them to select another time.
- **FR-4 (Table Combination Logic):** The system shall automatically calculate and combine adjacent smaller tables (e.g., two 4-seaters) if a requested Party Size exceeds standard individual table capacities. The system shall cap automatic table combination at a maximum of 12 Pax. Upon the conclusion of the reservation (status changed to 'Completed' or 'Cancelled'), the system shall automatically dissolve these logical links, returning each table to its individual 'Available' state.
- **FR-5 (Waitlist Management):** The system shall provide a digital waitlist for fully booked time slots and automatically notify the next queued customer if a cancellation occurs. The system shall grant a notified waitlist customer a 10-minute window to confirm the booking before automatically revoking the offer and notifying the next user in the queue. To prevent scheduling conflicts right before closing, this automated waitlist protocol shall be disabled 60 mins prior to the end of the operating shift, leaving any late-stage cancellations as manual walk-in availability.
- **FR-6 (Automated Notifications):** The system shall automatically dispatch booking confirmations, calendar invites (.ics), and upcoming reminders exclusively via SMTP Email integration.
- **FR-10 (Booking Cancellation):** The system shall allow customers to cancel upcoming reservations through their account dashboard. Upon cancellation, the system shall update the table status to "Available" and trigger the waitlist protocol. Cancellations made at least 24 hours in advance shall trigger an automated simulated deposit refund; cancellations within 2 hours of the booking shall be disabled in the portal, requiring the customer to contact the restaurant directly.

3.2.2 Restaurant Management System (Admin)

- **FR-7 (Visual Table Management):** The system shall provide administrators with a static, interactive floor plan grid reflecting the restaurant's actual layout, allowing staff to manually click and update real-time table statuses. The required UI color-coding shall be: Green (Available), Yellow/Amber (Reserved), Red (Occupied), and Grey (Dirty). The system shall automatically synchronize related states; if an Administrator manually changes a Table Status to 'Dirty', the system must automatically transition the associated Reservation Status from 'Seated' to 'Completed'.
- **FR-8 (Reservation & Shift Management):** The system shall allow administrators to view bookings in both list and calendar formats, manually enter phone/walk-in reservations, and block out dates for holidays or private events. The system shall enforce input validation for operating hours, strictly preventing administrators from setting a closing time that is earlier than the opening time, and preventing any bookings outside of defined operational shifts.
- **FR-9 (Guest CRM):** The system shall track and display individual customer metrics to the restaurant staff, including past dining history, VIP status, no-show counts, and specific allergy notes. A reservation shall be automatically flagged as a 'No-Show' in the CRM if the customer fails to check in within 15 minutes past their scheduled reservation time, which will subsequently trigger the deposit forfeiture workflow.
- **FR-11 (Menu Management):** The system shall allow administrators to upload, edit, or remove items from the digital menu displayed on the Customer Portal.
- **FR-12 (Manual Waitlist Control):** The system shall allow Restaurant Administrators to manually view, prioritize, and edit customers on the Waitlist Queue to accommodate VIPs or walk-in emergencies.
- **FR-13 (System Health Monitoring):** The Admin Dashboard shall display real-time connection status indicators for the system's three critical external dependencies (Supabase, Payment Gateway, and SMTP Email Server).

3.3 Use Case Model



3.3.1 Use Case #1 (U1): Book a Table & Pay Deposit

Author: LT1 Developers

Purpose: To allow a customer to search for, select, and successfully reserve a table, completing the transaction with a simulated deposit.

Requirements Traceability: Traces to FR-1, FR-2, FR-3, and FR-6.

Priority: High. This is the core functionality of the Minimum Viable Product (MVP).

Preconditions: The customer must be registered and logged into their account via the Supabase Auth system. The restaurant must have at least one table available for the requested time.

Post conditions: The Supabase database updates the selected table's status to "Reserved," and an automated confirmation is successfully dispatched to the user.

Actors: Customer, Supabase Backend, SMTP Email Server, Payment Gateway.

Extends: N/A

Flow of Events:

1. Customers enter their desired Date, Time, and Party Size (Pax).
2. System queries the Supabase database for real-time availability.
3. The system displays the available time slots.
4. The customer selects a time slot and reviews the view-only digital menu.
5. The customer enters payment details into the simulated checkout for the required deposit.
6. The system locks the table in the database to prevent double-booking.

Alternative Flow: If the requested time slot is fully booked, the system prompts the customer to trigger the "Join Virtual Waitlist" use case instead (Traces to FR-5).

Exceptions: If the simulated payment checkout fails, the booking process is aborted, the table is immediately unlocked in the database, and an error message is displayed to the customer.

Includes: U5 (Receive Automated Notifications)

Notes/Issues: *Real-time database locking via Supabase is critical in the Basic Flow to ensure two customers do not accidentally book the same table at the same millisecond.*

3.3.2 Use Case #2 (U2): Update Table Status via Static Grid

Author: LT1 Developers

Purpose: To allow a Restaurant Admin to manually update the real-time physical status of tables during a live service shift.

Requirements Traceability: Traces to FR-7.

Priority: High. This is the core functionality of the Minimum Viable Product (MVP).

Preconditions: The Restaurant Admin must be authenticated with Admin privileges.

Post conditions: The selected table's status is updated in the database and reflected in real time on the floor plan grid.

Actors: Restaurant Admin, Supabase Backend

Extends: N/A

Flow of Events:

1. Admin navigates to the Visual Table Management dashboard.
2. The system displays the static, interactive floor plan grid reflecting current database statuses.
3. Admin clicks on a specific table icon (e.g., Table 4).
4. Admin selects a new status from the dropdown menu (e.g., changing from "Reserved" to "Occupied" or "Dirty").
5. The system saves the status to Supabase and visually updates the color-coding of the table on the grid.

Alternative Flow: N/A

Exceptions: If the restaurant loses its internet connection, the system displays an "Offline Mode" warning, preventing the admin from clicking or changing statuses until the connection to Supabase is restored.

Includes: N/A

Notes/Issues: *Real-time database locking via Supabase is critical in the Basic Flow to ensure two customers do not accidentally book the same table at the same millisecond.*

3.3.3 Use Case #3 (U3): Join Virtual Waitlist

Author: LT1 Developers

Purpose: To allow a customer to join a digital queue when their desired time slot is fully booked.

Requirements Traceability: Traces to FR-5, FR-6.

Priority: Medium.

Preconditions: The customer must be logged in. The requested time slot must have zero available tables.

Post conditions: The customer is added to the Supabase waitlist database table, and an email notification is sent if a table opens up.

Actors: Customer, Supabase Backend.

Extends: N/A

Flow of Events:

1. Customer attempts to search for a fully booked time slot.
2. The system displays a "No Tables Available" message and offers the "Join Waitlist" button.
3. The customer clicks the button and confirms their party size.
4. The system logs the request in the Supabase backend with a timestamp.

Alternative Flow: If a reserved customer cancels their booking, the system automatically triggers an email to the first person on this waitlist.

Exceptions: If the waitlist for that specific night exceeds maximum capacity (e.g., 50 parties), the system disables the button and displays "Waitlist Full."

Includes: N/A

Notes/Issues: N/A

3.3.4 Use Case #4 (U4): Manage Account & Login

Author: LT1 Developers

Purpose: To allow a customer to create an account, log in securely, and manage their dietary preferences.

Requirements Traceability: Traces to FR-1.

Priority: High.

Preconditions: The user must have a valid email address.

Post conditions: A secure session token is generated, granting the user access to the booking engine.

Actors: Customer, Supabase Backend.

Extends: N/A

Flow of Events:

1. The user navigates to the login/register page.
2. Users enter their email, password, and basic contact info.
3. Supabase Auth processes the credentials and generates a secure token.
4. Users access their profile dashboard to add specific dietary restrictions (e.g., "Peanut Allergy").

Alternative Flow: User requests a password reset link via email.

Exceptions: If the user enters an invalid email format or incorrect password, the system displays an authentication error.

Includes: N/A

Notes/Issues: All password hashing and security compliance is handled entirely by Supabase Auth, reducing liability for the LT1 developers.

3.3.5 Use Case #5 (U5): Manage Reservations & Shifts

Author: LT1 Developers

Purpose: To give restaurant staff the ability to view all daily bookings, log walk-ins, and block out dates.

Requirements Traceability: Traces to FR-8.

Priority: High.

Preconditions: The user must be logged in with "Restaurant Admin" privileges.

Post conditions: The master calendar is updated in the database.

Actors: Restaurant Admin, Supabase Backend.

Extends: N/A

Flow of Events:

1. Admin navigates to the Master Calendar dashboard.
2. System retrieves all upcoming bookings from Supabase.
3. Admin clicks "Add Walk-in" and manually types in a customer's name and party size.
4. The system assigns the walk-in to an available table and locks it.

Alternative Flow: Admin selects a future date (e.g., Christmas) and clicks "Block Date" to prevent any online customers from booking.

Exceptions: N/A

Includes: N/A

Notes/Issues: N/A

3.3.6 Use Case #6 (U6): View Guest CRM Data

Author: LT1 Developers

Purpose: To allow staff to view a customer's dining history and allergy notes to provide personalized service.

Requirements Traceability: Traces to FR-9.

Priority: Medium.

Preconditions: The user must be logged in with "Restaurant Admin" privileges.

Post conditions: CRM data is displayed securely on the dashboard.

Actors: Restaurant Admin, Supabase Backend.

Extends: N/A

Flow of Events:

1. Admin clicks on an upcoming reservation in the daily list.
2. The system fetches the Customer's CRM profile from Supabase.
3. The system displays the customer's total past visits, no-show count, and any allergy notes (e.g., "Gluten-Free").

Alternative Flow: Admin manually edits the CRM note to add a VIP tag after a good service shift.

Exceptions: If it is a guest's first time booking, the CRM profile simply displays "First Time Guest."

Includes: N/A

Notes/Issues: *This fulfills the specific stakeholder request to track customer history without needing a massive enterprise SaaS framework.*

3.3.7 Use Case #7 (U7): Cancel Reservation

Author: LT1 Developers

Purpose: To allow a customer to cancel an upcoming reservation, freeing up the table for the restaurant and triggering the virtual waitlist.

Requirements Traceability: Traces to FR-10, FR-5, and FR-6.

Priority: High, Essential for resolving the waitlist automation logic.

Preconditions: The customer must be logged into their account via Supabase Auth and have at least one active, upcoming reservation.

Post conditions: The Supabase database updates the reservation status to "Cancelled." The table becomes available, and automated emails are dispatched.

Actors: Customer, Supabase Backend, SMTP Email Server

Extends: N/A

Flow of Events:

1. The customer navigates to their profile dashboard.
2. The system retrieves and displays the customer's upcoming reservations from the Supabase database.
3. The customer selects a specific reservation and clicks "Cancel Booking."
4. The system prompts for a final confirmation, reminding the user of the simulated deposit policy and notifying them of refund eligibility (if >24 hours prior).
5. The customer confirms. The system updates the reservation status to "Cancelled" and removes the database lock on that table. The system dispatches a cancellation confirmation email via the SMTP server.

Alternative Flow: If there are users currently on the Virtual Waitlist for that specific Date and Time, the system automatically dispatches an email notification to the first person in the queue.

Exceptions: If the customer attempts to cancel within 2 hours of the reservation time, the system disables the cancellation button and prompts them to call the restaurant directly.

Includes: U5 (Receive Automated Notifications)

Notes/Issues: N/A

4 Other Non-functional Requirements

4.1 Performance Requirements

PR-1 (Response Time): The web application shall load the customer-facing real-time availability grid and the administrator's static floor plan within 3 seconds over a standard 4G/LTE or broadband internet connection.

Fast load times prevent customer abandonment during the booking process.

PR-2 (Real-Time Concurrency): The system shall utilize Supabase's real-time database row-locking mechanisms to resolve conflicts within 1 second. In the event of a conflict, the user who loses the reservation 'race' shall be presented with a 'Table No Longer Available' notification, and their availability grid shall automatically refresh to show current openings.

Strict timing on concurrency prevents catastrophic double-booking scenarios.

PR-3 (Notification Latency): Automated booking confirmations shall be dispatched to the SMTP server within 10 seconds of a successful simulated deposit checkout.

Immediate confirmation reassures the customer that their payment and booking were successfully processed.

4.2 Safety and Security Requirements

Safety Requirements (Safeguards against operational loss):

- **SAF-1 (Data Loss Prevention):** The system shall rely on Supabase's automated Point-in-Time Recovery (PITR) and daily backups to prevent the catastrophic loss of guest CRM data and future reservations in the event of a database failure.
- **SAF-2 (Offline Failsafe):** If the restaurant's operational dashboard loses its internet connection, the system must clearly display an "Offline Warning" to the staff. It must disable the ability to change table statuses until the connection is restored to prevent staff from making operational decisions based on desynchronized data.

Security Requirements (Protection of privacy and access):

- **SEC-1 (Identity and Authentication):** All system access shall require secure authentication managed via Supabase Auth. The system must strictly separate Customer accounts from Restaurant Administrator accounts using Role-Based Access Control (RBAC).
- **SEC-2 (Connection Security):** To protect mobile and desktop connections, all web traffic between the client browsers and the Supabase backend shall be strictly encrypted using standard HTTPS/TLS protocols.

- **SEC-3 (Payment Privacy):** To mimic PCI-DSS compliance for the MVP, the system shall not store raw credit card numbers in the local database. All deposit transactions must be handled via a secure, tokenized simulated checkout gateway.

4.3 Software Quality Attributes

4.3.1 Usability

Because the system serves a diverse demographic of restaurant customers, ease of use is prioritized over a steep learning curve.

- **Requirement:** The customer-facing booking engine shall allow a user to complete a reservation in no more than four steps (Search, Select Time, Input Details, Confirm Deposit).
- **Implementation Plan:** This will be achieved by utilizing a clean, responsive web interface with intuitive dropdowns for Date and Party Size, ensuring the interface clearly guides the user without requiring external instructions or tutorials.

4.3.2 Maintainability

The system must be designed for change, specifically regarding the physical layout of the restaurant.

- **Requirement:** The system shall allow developers to update the static floor plan grid without completely rewriting the core booking logic.
- **Implementation Plan:** This will be achieved by using a modular, component-based frontend architecture. By decoupling the visual floor plan grid from the Supabase reservation database queries, LT1 developers can quickly add or remove table icons if Gordon Ramsay Restaurant undergoes physical renovations.

4.3.3 Availability

The reservation system must remain accessible outside of standard restaurant operating hours so customers can book tables 24/7.

- **Requirement:** The web application and database APIs shall target an industry-standard 99.9% uptime.
- **Implementation Plan:** Instead of relying on a local, on-premise server situated inside the restaurant (which is vulnerable to local power outages), this high availability is achieved by hosting the core database and authentication logic on Supabase's globally distributed, managed cloud infrastructure.

5 Other Requirements

5.1 Database Requirements

- **DB-1 (Relational Integrity):** The system shall utilize a PostgreSQL relational database (hosted via Supabase) to ensure ACID (Atomicity, Consistency, Isolation, Durability) compliance. This guarantees that if a reservation transaction is interrupted midway, the database will not save a partial or corrupted booking.
- **DB-2 (Data Retention):** Customer profiles and Guest CRM data shall be retained securely in the database indefinitely, unless a customer explicitly requests the deletion of their account.
- **DB-3 (Time Zone Standardization):** The database shall store all reservation date and time data strictly in UTC standard format, and automatically convert it to the restaurant's local time zone for display on the Customer Portal and Admin Dashboard.

5.2 Legal and Compliance Requirements

- **LEG-1 (Data Privacy):** The system shall comply with Republic Act No. 10173 (Data Privacy Act of 2012) regarding the collection and storage of Personally Identifiable Information (PII). Users must be presented with a mandatory consent checkbox regarding the storage of their email addresses and dietary restrictions during account registration. Furthermore, to comply with the Right to Erasure, the Customer Portal shall include an automated 'Delete Account' function that immediately and permanently purges all associated PII and Guest CRM data upon user request.
- **LEG-2 (Payment Processing Liability):** To mitigate legal liability for the LT1 developer team during the MVP phase, the system shall not capture, store, process, or transmit any real credit card Primary Account Numbers (PAN) during the MVP phase. The deposit functionality shall operate exclusively as a simulated checkout environment during the MVP phase. In any future production deployment involving real financial transactions, the system shall integrate a fully PCI-DSS-compliant third-party payment gateway (e.g., Stripe or PayMongo).

Appendix A – Data Dictionary

Data Item / Variable	Type	Description	Possible Values/States	Related Operations (Use Cases)	Related Requirements
User Role	Constant	Defines the permission and access level of a user authenticated via Supabase Auth.	Customer, Restaurant Admin	U4 (Manage Account & Login), U5 (Manage Reservations)	FR-1, SEC-1
Table Status	State Variable	Represents the real-time physical availability of a specific table on the static floor plan grid.	Available, Reserved, Occupied, Dirty	U2 (Update Table Status via Static Grid)	FR-7, PR-1
Reservation Status	State Variable	The lifecycle phase of a customer's specific booking.	Pending, Confirmed, Seated, Completed, No-Show, Cancelled	U1 (Book Table & Pay Deposit), U5 (Manage Reservations)	FR-3, FR-8
Party Size (Pax)	Input	The number of guests requested by the customer during the availability search.	Integer (e.g., 1 to 20+)	U1 (Book Table & Pay Deposit)	FR-2, FR-4

Desired Date & Time	Input	The specific calendar day and time block the customer wishes to book.	Format for Date and Time (e.g., YYYY-MM-DD HH:MM)	U1 (Book Table & Pay Deposit), U3 (Join Waitlist)	FR-2, FR-3
Dietary Restrictions	Input/State	Notes about a customer's health, allergies, or preferences that are connected to their CRM profile.	Text String (e.g., Peanut Allergy, Gluten-Free, None)	U4 (Manage Account), U6 (View Guest CRM Data)	FR-1, FR-9
Waitlist Position	State Variable	The sequential order of a customer waiting for a cancellation on a fully booked date.	Integer (e.g., 1, 2, 3) or Null	U3 (Join Virtual Waitlist)	FR-5
Waitlist Status	State Variable	Tracks the lifecycle of a waitlist entry.	Pending, Notified, Confirmed, Expired	U3 (Join Waitlist), U7 (Cancel Reservation)	FR-5
Booking Confirmation	Output	The automated SMTP payload is dispatched upon successful deposit.	JSON/HTML Payload containing Reservation details and .ics invite	U1 (Book Table & Pay Deposit), U5 (Receive Notifications)	FR-6, PR-3

Appendix B - Group Log

The following log details the internal team meetings, client negotiations, and architectural discussions my team and I conducted to produce this Software Requirements Specification.

Meeting 1: Initial Requirements Analysis & Scope Review

Date: February 2, 2026

Attendees: Qdreon (LT1 Developer Team)

Topics Discussed: We reviewed the initial client wishlist and our first draft of the functional requirements.

Outcomes & Decisions: *We noticed right away that we had some severe "scope creep" in our initial draft, specifically regarding the "Multi-Tenant Super Admin" dashboard and the complex "Drag-and-Drop Floor Plan" builder.*

- *We decided as a group to schedule a negotiation meeting with the client stakeholders.*
- *Our goal was to scale the project down to a realistic Minimum Viable Product (MVP - Version 1.1) tailored specifically for a single Gordon Ramsay Restaurant location.*

Meeting 2: Stakeholder Negotiation & Technical Planning (Recorded Session 1)

Date: February 5, 2026

Attendees: Qdreon (LT1 Developer Team) and LT3 Stakeholder Team

Topics Discussed: Proposing our Version 1.1 MVP limitations to ensure we could build and deliver the system on time.

Outcomes & Decisions:

- **Approved:** *We successfully negotiated dropping the complex "drag-and-drop" canvas in favor of a static, interactive floor plan grid.*
- **Approved:** *We agreed to implement automated SMTP Email notifications instead of premium SMS push notifications to save budget.*
- **Approved:** *We officially locked the scope to one restaurant location, eliminating the Super Admin tenant manager requirement.*

Meeting 3: Architecture Finalization & Use Case Deep-Dive (Recorded Session 2)

Date: February 9, 2026

Attendees: Qdreon (LT1 Developer Team) and LT3 Stakeholder Team

Topics Discussed: Finalizing our tech stack and defining the formal UML Use Cases based on stakeholder feedback.

Outcomes & Decisions:

- *To meet our MVP deadline, we officially chose **Supabase** for our backend architecture to handle database hosting and authentication.*
- *We decided to use strict UML standard modeling for our Use Case Diagrams to ensure compliance with COMET methodologies.*

Meeting 4: SRS Document Compilation & Final Review

Date: February 15, 2026

Attendees: Qdreon (LT1 Developer Team)

Topics Discussed: Finalizing sections 3 and 4 of our IEEE SRS template.

Outcomes & Decisions:

- *We drafted 6 core Use Cases, mapping out both Customer and Admin pathways.*
- *We finalized Performance and Security requirements, specifically noting Supabase's role in concurrency locking.*
- *We finalized the Appendix A Data Dictionary to define all backend variables for the Supabase schema.*